15

Stored Procedures

15.1 Programmabläufe speichern

Beim Einsatz von Datenbanksystemen spielt der Datenbankserver eine herausragende Rolle. Er arbeitet am zentralen Speicherort der Daten, verwaltet die Daten dort und liefert auf Anfragen die entsprechenden Ergebnisse. Durch diese zentrale Datenhaltung wird ein kontrollierter Zugang zu den Daten gewährleistet.

Damit der Server seiner Aufgabe als zentrale Verarbeitungseinheit gerecht wird, sollten alle aufwendigen Arbeiten mit den Daten direkt auf dem Server durchgeführt werden. Die Clients erhalten nur die Ergebnisdaten und werden dadurch entlastet, dass sie keine Berechnungen durchführen müssen.

Für diesen Zweck ist es notwendig, dass ganze Abläufe von SQL-Anweisungen auf dem Datenbankserver ausgeführt werden und auch die Bearbeitung möglicher Zwischenschritte hier erfolgt. In SQL können Sie dafür sogenannte Prozeduren oder Stored Procedures (gespeicherte Prozeduren) einsetzen, mit denen Programmabläufe vereinfacht und automatisiert werden können.

In der Praxis werden die Daten oft nicht nur auf einem Datenbankserver gehalten, sondern z. B. durch Replikation (Daten in mehreren Datenbanken auf dem gleichen Stand halten) und Shadowing (Kopie der Datenbank) auf mehreren Servern verteilt. Außerdem können Unternehmensdaten auf mehrere Server verteilt sein (z. B. Filialen einer Sparkasse), sodass es keinen gemeinsamen Datenbestand gibt.

Prozeduren auf dem Server speichern

Stored Procedures sind auf dem Server gespeicherte **Prozeduren** (Unterprogramme), in denen neben den üblichen SQL-Anweisungen auch zusätzliche Befehle zur Ablaufsteuerung und Auswertung von Bedingungen zur Verfügung stehen. Sie sind damit den Makros mancher Anwendungsprogramme vergleichbar, mit denen Sie Programmabläufe automatisieren können.

© HERDT-Verlag
Schulversion

Beispiel

Bei der Arbeit mit einem Datenbanksystem werden meist verschiedene Daten abgefragt und verarbeitet. Bei der Verarbeitung werden aber oft die gleichen Arbeitsabläufe, jedoch mit unterschiedlichen Werten und/ oder Parametern ausgeführt. Diese Arbeitsabläufe können in Stored Procedures zusammengefasst werden. Bei einer Banküberweisung werden z. B. der Betrag und die Informationen zu den beiden Konten benötigt. Sie können diese vom Client (Bankautomat) an eine Stored Procedure auf dem Server übergeben. Diese führt die notwendigen Operationen aus und liefert als Rückgabewert eine Information zum Erfolg oder Misserfolg der Überweisung.

Höhere Verarbeitungsgeschwindigkeit

Ein wichtiger Vorteil der Stored Procedures liegt in einem Geschwindigkeitsgewinn bei der Ausführung umfangreicher Datenbearbeitungen. Dieser liegt vor allem darin begründet, dass die Prozeduren auf dem Server ausgeführt werden und daher direkten Zugriff auf die gespeicherten Daten erhalten. Ein oft sehr zeitaufwendiger Transport der Daten über ein Netzwerk entfällt. Je langsamer das Netzwerk ist und je größer der Datenbestand, umso deutlicher wird dabei der Gewinn an Geschwindigkeit. Die Verringerung der Datenmenge bei der Datenübertragung über das Netzwerk kann bei vielen aktiven Client-Anwendungen sehr bedeutsam sein.

Bessere Systemsicherheit

Neben der höheren Geschwindigkeit spricht auch eine bessere Sicherheit des Datenbanksystems für den Einsatz von Stored Procedures. Mit ihrer Hilfe ist es möglich, den Anwender einer Datenbank von vielen systemrelevanten Daten abzuschirmen. Er erfährt nicht die Namen der Tabellen und deren Struktur und er erhält keinen Zugriff auf möglicherweise sensible Daten, die bei der Verarbeitung durch gespeicherte Prozeduren verwendet werden.

Thin Clients

Um Updates und Programmänderungen zu vereinfachen, wird die Funktionalität der Anwendungen vom Client mehr auf den Server verlagert. Die Thin Clients (thin = dünn) besitzen dann nur wenig Funktionalität, z. B. eine Benutzeroberfläche zur Darstellung der Resultate. Auf dem Server werden alle gemeinsam genutzten Funktionen, die sogenannte Geschäftslogik (Business Logic), bereitgestellt. Sind nun beispielsweise Updates dieser Funktionen notwendig, müssen diese nur zentral auf dem Server aktualisiert werden (bei gleichbleibenden Schnittstellen), anstatt auf vielen Tausend Clients.

Durch die Kapselung von SQL-Anweisungen und Verarbeitungsschritten in einzelne Prozeduren wird zusätzlich die Wartung der Programmcodes vereinfacht. Für den Anwender sind nur der Aufruf einer Stored Procedure und deren Rückgabewerte von Bedeutung. Die eigentlichen Anweisungen können unabhängig davon optimiert und verändert werden.

Schulversion
© HERDT-Verlag

Ausführbare und selektierbare Prozeduren

Die Implementation der Stored Procedures ist bei jedem Datenbanksystem verschieden. Manche Datenbanksysteme unterstützen neben den ausführbaren Prozeduren als Besonderheit selektierbare Prozeduren.

Ausführbare Prozeduren (`CALL`-Prozedur)	Sie werden aufgerufen, um Änderungen an den Datenbeständen vorzunehmen. In der Regel erfolgt die Rückgabe eines Statuswertes, der über Erfolg oder Misserfolg der Aktion informiert. Es können auch mehrere Werte zurückgeliefert werden, um z. B. die Ergebnisse einer Berechnung zu übermitteln.
Selektierbare Prozeduren (`SELECT`-Prozedur)	Diese Prozeduren liefern eine Ergebnismenge als Resultat und können in einer `SELECT`-Abfrage wie eine Tabelle verwendet werden. Sie können auch mit Tabellen, Views oder anderen Stored Procedures verknüpft werden.

- Das Datenbanksystem PostgreSQL unterstützt keine Stored Procedures im klassischen Sinne. Es bietet jedoch eine breite Unterstützung zur Programmierung von benutzerdefinierten Funktionen. Dazu stellt PostgreSQL in der Standardimplementierung vier prozedurale Sprachen bereit: PL/pgSQL, PL/Tcl, PL/Perl und PL/Python. Sie können in diesen Sprachen mit der Anweisung `CREATE FUNCTION` in PostgreSQL Prozeduren entsprechend umschreiben. Weitere Informationen finden Sie im Handbuch von PostgreSQL.
- Jedes Datenbanksystem, das Stored Procedures unterstützt, besitzt eine eigene Implementierung von Stored Procedures, die sich in der Deklaration und Verwendung der Anweisungen unterscheidet. Die grundsätzliche Arbeitsweise und Verwendung ist jedoch bei allen gleich.
- Im Folgenden werden Stored Procedures mit MariaDB erstellt und verwendet.

15.2 Stored Procedures erstellen und bearbeiten

Prozedur definieren

Bevor Sie eine Stored Procedure verwenden können, muss sie in einer Datenbank definiert werden. Dies geschieht nach dem Verbinden mit der Datenbank mit der Anweisung `CREATE PROCEDURE`. Bei der Deklaration werden neben einem Prozedurnamen und den eigentlichen Anweisungen auch die Übergabe- und Rückgabeparameter festgelegt.

Zur besseren Unterscheidung von anderen Elementen der Datenbank kann dem Namen der Prozedur das Präfix *p_* vorangestellt werden.

Beispiel: *EinfacheProzedur.sql*

Es sollen alle Artikelnamen mit den Namen der dazugehörigen Lieferanten ausgewählt werden. Hierzu müssen die Artikelnamen aus der Tabelle *t_artikel* und die Lieferantennamen aus der Tabelle *t_liefer* ermittelt und verknüpft werden.

© HERDT-Verlag Schulversion

```
① DELIMITER //
② CREATE PROCEDURE p_artikel_liefer ()
③ BEGIN
④ SELECT t_artikel.name , t_liefer.name
  FROM t_artikel INNER JOIN t_liefer ON
  t_artikel.lieferant=t_liefer.id;
⑤ END //
⑥ DELIMITER ;
⑦ CALL p_artikel_liefer ();
```

① Innerhalb der Prozedur werden SQL-Anweisungen wie üblich mit einem Semikolon abgeschlossen. Damit die Eingabe an dieser Stelle aber nicht fälschlicherweise abgebrochen wird, muss mit dem Befehl DELIMITER ein neues Endekennzeichen für SQL-Anweisungen definiert werden. Hier ist es das Zeichen //.

② An dieser Stelle beginnt die Definition der Stored Procedure mit dem Namen *p_artikel_liefer*. Danach werden in runden Klammern die Namen der Übergabeparameter und deren Datentypen angegeben. Unter diesen Namen können die übergebenen Werte im Verlauf der Prozedur angesprochen werden. Im Beispiel sind keine Übergabeparameter notwendig.

③ Einzelne Blöcke werden stets durch das Schlüsselwort BEGIN eingeleitet und mit END abgeschlossen.

④ Hier beginnt der Anweisungsblock der Prozedur. Mithilfe einer SELECT-Abfrage werden die Artikel und die dazugehörigen Lieferantennamen aus den entsprechenden Tabellen ausgewählt.

⑤ Die Deklaration der Prozedur endet mit END. Die Anweisung muss entsprechend der Neudefinition des Endekennzeichens (①) mit dem Zeichen // abgeschlossen werden.

⑥ Nun kann als Endekennzeichen wieder das Semikolon definiert werden. Achten Sie dabei auf das Leerzeichen zwischen dem Befehl DELIMITER und dem Semikolon.

⑦ Mit der Anweisung CALL wird die Prozedur ausgeführt.

Syntax einfacher Prozeduren

```
CREATE PROCEDURE prozedurname ([ IN | OUT | INOUT] parameter1
datentyp1, ...)
   BEGIN
    anweisungsblock
   END;
```

- Die Anweisung beginnt mit den Schlüsselwörtern CREATE PROCEDURE. Danach folgt der Prozedurname.
- In runden Klammern werden die einzelnen Übergabeparameter der Prozedur angegeben. Bei mehreren Parametern müssen diese durch Kommata getrennt werden. Für jeden Parameter muss der gewünschte Datentyp angegeben werden.
- Der Vorgabewert für Parameter ist der Eingabeparameter IN. Das bedeutet, die Daten können von der Prozedur nur ausgelesen werden. Der Ausgabeparameter OUT übergibt Werte an die CALL-Anweisung. Der INOUT-Parameter kann beides. Um den Vorgabewert zu ändern, setzen Sie das Schlüsselwort OUT oder INOUT vor den Parameternamen.
- In einem durch BEGIN und END eingeschlossenen Anweisungsblock folgt die Liste der Anweisungen, die von der Prozedur ausgeführt werden sollen.

© HERDT-Verlag

Endekennzeichen festlegen

Die einzelnen Anweisungen der Stored Procedure müssen, wie alle SQL-Anweisungen, mit einem Semikolon als Endekennzeichen abgeschlossen werden. Erfolgt die Eingabe der Prozedur unter MariaDB mit einem Standard-Client, wird das Semikolon jedoch fälschlich als Ende der gesamten Prozedurdefinition interpretiert. Um dies zu vermeiden, können Sie mit der Anweisung `DELIMITER` *`endekennzeichen`* temporär ein anderes Zeichen als Endekennzeichen (als sogenannten Terminator) festlegen.

Verwenden Sie als Endekennzeichen ein Zeichen, das im Normalfall in keiner SQL-Anweisung vorkommt.

Damit das Semikolon wieder als Endekennzeichen interpretiert wird, müssen Sie die Anweisung `DELIMITER ';'` nach der Definition der Stored Procedure erneut aufrufen.

Syntax

```
DELIMITER endekennzeichen
```

- Die Anweisung beginnt mit dem Schlüsselwort `DELIMITER`. Danach folgt das gewünschte Zeichen, das ab diesem Zeitpunkt als Endekennzeichen verwendet werden soll.
- Beim Beenden der Anweisung muss stets das aktuelle Endekennzeichen verwendet werden, standardmäßig das Semikolon.

Anweisungsblöcke definieren

Bei der Definition der Stored Procedure wird mit den Anweisungen `BEGIN` und `END` ein Anweisungsblock definiert. Die Anweisungen werden auch im Verlauf der Prozedur benötigt, um zusammengehörige Anweisungen zu einem Block zusammenzufassen.

- In allen Anweisungen, die der Programmsteuerung dienen, beispielsweise einer Alternative (`IF... THEN... ELSE...`), können eine oder mehrere SQL-Anweisungen ausgeführt werden. Sind mehrere Anweisungen erforderlich, müssen diese durch die Schlüsselwörter `BEGIN` und `END` eingeschlossen werden. Es wird dann von einem Anweisungsblock gesprochen.
- Anweisungsblöcke können beliebig ineinander verschachtelt werden.

Syntax

```
BEGIN
   anweisung1;
   ...
END
```

- Ein Anweisungsblock wird mit dem Schlüsselwort `BEGIN` eingeleitet und mit `END` abgeschlossen. Dazwischen können beliebige Anweisungen angegeben werden.

© HERDT-Verlag
Schulversion

Vorhandene Prozeduren anzeigen

Mit der Anweisung `SHOW PROCEDURE STATUS` erhalten Sie eine Auflistung der vorhandenen Stored Procedures der Datenbank in Tabellenform. Da die Tabellenform für diese Anzeige etwas unübersichtlich ist, können Sie auch die Listenform wählen, indem Sie `SHOW PROCEDURE STATUS \G;` eingeben. Zusätzlich können Sie die Anzeige durch eine `WHERE`-Klausel auf eine Datenbank beschränken.

```
MariaDB [uebungen]> SHOW PROCEDURE STATUS WHERE db="uebungen" \G;
*************************** 1. row ***************************
                  Db: uebungen
                Name: p_artikel_liefer
                Type: PROCEDURE
             Definer: root@localhost
            Modified: 2021-06-10 17:16:32
             Created: 2021-06-10 17:16:32
       Security_type: DEFINER
             Comment:
character_set_client: latin1
collation_connection: latin1_swedish_ci
  Database Collation: latin1_swedish_ci
1 row in set (0.044 sec)
```

Anzeige der definierten Stored Procedures in Listenform

Syntax

```
SHOW PROCEDURE STATUS;
```

- Die Anweisung besteht aus den Schlüsselwörtern `SHOW PROCEDURE STATUS`. Als Resultat liefert sie eine Liste der definierten Prozeduren.

Mit der Anweisung `SHOW CREATE PROCEDURE` *`prozedurname`* können Sie sich den Code einer bestimmten Prozedur anzeigen lassen.

Prozeduren löschen

Wenn eine Prozedur nicht wunschgemäß arbeitet oder wenn sie nicht mehr benötigt wird, können Sie die Prozedur löschen. Dies geschieht mit der Anweisung `DROP PROCEDURE`.

Eine Prozedur kann nur dann gelöscht werden, wenn Sie momentan nicht ausgeführt wird und andere Stored Procedures nicht darauf aufbauen.

Syntax

```
DROP PROCEDURE prozedurname;
```

- Die Anweisung beginnt mit den Schlüsselwörtern `DROP PROCEDURE`. Danach folgt der Name der zu löschenden Prozedur.

Prozeduren bearbeiten

Eine umfangreiche Stored Procedure arbeitet selten von Anfang an ohne Fehler. Statt eine Prozedur im Falle eines Fehlers zu löschen und neu zu erstellen, können Sie die Prozedur mit der Anweisung `ALTER PROCEDURE` bearbeiten. Auf diese Weise können Sie eine Prozedur schrittweise weiterentwickeln.

© HERDT-Verlag

Mithilfe der Anweisung ALTER PROCEDURE können Sie den Anweisungsteil der Prozedur bearbeiten, ohne Übergabe- und Rückgabeparameter zu verändern. Die Syntax entspricht dabei der Anweisung CREATE PROCEDURE.

Syntax

```
ALTER PROCEDURE prozedurname ([ IN | OUT | INOUT] parameter1
datentyp1, ...)
   BEGIN
    anweisungsblock
   END;
```

- Die Anweisung beginnt mit den Schlüsselwörtern ALTER PROCEDURE. Danach folgt der Prozedurname.
- In runden Klammern werden die einzelnen Übergabeparameter der Prozedur angegeben.
- In einem durch BEGIN und END eingeschlossenen Anweisungsblock folgt die Liste der Anweisungen, die von der Prozedur ausgeführt werden sollen.

Um bei umfangreicheren Prozeduren den Überblick nicht zu verlieren, ist es praktischer, diese mithilfe eines Editors zu erstellen und dann über die Zwischenablage zu kopieren.

15.3 Beispielanwendung für Stored Procedures

Eine komplexe Prozedur erstellen

In einer Stored Procedure können Sie die SQL-Anweisungen SELECT, INSERT, DELETE und UPDATE verwenden. Zusätzlich existieren noch Anweisungen zur Steuerung des Programmablaufs, die nur innerhalb einer Prozedur erlaubt sind. Auch Variablen können für den internen Gebrauch in einer Prozedur definiert werden. Hier ein Beispiel:

Beispiel: *CreateProcedure.sql*

Es soll ein neuer Mitarbeiter in die Tabelle *t_ma* eingefügt werden. Dieser Mitarbeiter wird gleichzeitig einem Projekt zugeordnet. Dieses Projekt ist entweder bereits in der Tabelle *t_proj* enthalten oder die Prozedur erstellt einen neuen Eintrag. Danach soll die Prozedur in der Tabelle *t_ma_pro* einen neuen Eintrag mit dem Mitarbeiter und dem zugehörigen Projekt erstellen.

```
① DELIMITER //
② CREATE PROCEDURE p_neuer_ma
    (name VARCHAR(50), vorname VARCHAR(50), strasse VARCHAR(150),
     hnr VARCHAR(5), plz VARCHAR(5), ort VARCHAR(50), land
VARCHAR(4),
     projektname VARCHAR(50))
③ BEGIN
④     DECLARE mid INTEGER;
      DECLARE pid INTEGER;
```

```
⑤     SELECT MAX(id) + 1 FROM t_ma INTO mid;
⑥     INSERT INTO t_ma (id, name, vname, str, hnr, plz, ort, land)
          VALUES(mid, name, vorname, strasse, hnr, plz, ort, land);
⑦     SELECT COUNT(id) FROM t_proj WHERE t_proj.name = projektname
        INTO pid;
⑧     IF (pid = 0) THEN
         SELECT MAX(id) + 1 FROM t_proj INTO pid;
         INSERT INTO t_proj (id, name) VALUES(pid, projektname);
      ELSE
         SELECT id FROM t_proj WHERE t_proj.name = projektname
           INTO pid;
      END IF;
⑨     INSERT INTO t_ma_proj (ma_id, proj_id)
         VALUES(mid, pid);
⑩  END //
⑪  DELIMITER ;
⑫  CALL p_neuer_ma
      ('Herold', 'Frank', 'Waldstr.', '24', '10115', 'Berlin', 'D',
       Kundenumfrage');
⑬  CALL p_neuer_ma
      ('Teschau', 'Ines', '', '', '1210', 'Wien', 'AT',
   'Kundenumfrage');
```

① Innerhalb der Prozedur werden SQL-Anweisungen wie üblich mit einem Semikolon abgeschlossen. Damit die Eingabe an dieser Stelle aber nicht fälschlicherweise abgebrochen wird, muss mit dem Befehl `DELIMITER` ein neues Endekennzeichen für SQL-Anweisungen definiert werden. Hier ist es das Zeichen `//`.

② An dieser Stelle beginnt die Definition der Stored Procedure mit dem Namen *p_neuer_ma*. Danach werden in runden Klammern die Namen der Übergabeparameter und deren Datentypen angegeben. Unter diesen Namen können die übergebenen Werte im Verlauf der Prozedur angesprochen werden.

③ Einzelne Blöcke werden stets durch das Schlüsselwort `BEGIN` eingeleitet und mit `END` abgeschlossen. Hier beginnt der Anweisungsblock der Prozedur.

④ Zuerst werden die lokalen Variablen `mid` und `pid` für den Gebrauch innerhalb der Prozedur definiert.

⑤ Mithilfe einer `SELECT`-Abfrage wird die größte Mitarbeiter-ID ermittelt und um den Wert 1 erhöht. Das Ergebnis wird mithilfe des Schlüsselwortes `INTO` in der Variablen `mid` gespeichert.

⑥ Die eben ermittelte Mitarbeiter-ID wird verwendet, um einen neuen Datensatz in der Tabelle *t_ma* zu erzeugen. Dabei werden zusätzlich die Übergabeparameter der Prozedur verwendet.

⑦ Die Abfrage ermittelt, ob es für das Projekt *projektname* einen Datensatz in der Tabelle *t_proj* gibt. Der Spaltenname *name* muss in der `WHERE`-Klausel mit dem Tabellennamen *t_proj* qualifiziert werden, damit ihn das System vom gleichnamigen Parameter *name* unterscheidet. Das Ergebnis der Abfrage wird in der Variablen `pid` gespeichert. Wird ein Datensatz gefunden, ist der Wert von `pid` gleich 1, sonst 0.

© HERDT-Verlag

⑧ Mit einer `IF`-Anweisung wird an dieser Stelle überprüft, ob `pid` den Wert 0 enthält. In diesem Fall muss ein neuer Datensatz erzeugt werden. Dafür wird mit einer Abfrage eine neue Projekt-ID ermittelt und der Datensatz wird eingefügt. Existiert schon ein passender Datensatz, wird im `ELSE`-Zweig die entsprechende Projekt-ID in einer Abfrage ausgelesen und in der Variablen `pid` gespeichert.

⑨ Mit einer `INSERT`-Anweisung für die Tabelle *t_ma_proj* wird der Mitarbeiter mit dem entsprechenden Projekt in Bezug gesetzt. Dafür werden die entsprechenden ID-Nummern in den Variablen `pid` und `mid` verwendet.

⑩ Die Deklaration der Prozedur endet mit `END`. Die Anweisung muss entsprechend der Neudefinition des Endekennzeichens (①) mit dem Zeichen `//` abgeschlossen werden.

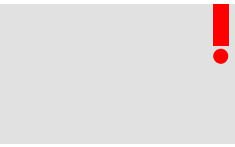
⑪ Nun kann als Endekennzeichen wieder das Semikolon definiert werden.

⑫ Mit der Anweisung `CALL` wird die Prozedur ausgeführt. Hier wird ein neuer Mitarbeiter eingefügt, der am Projekt *Kundenumfrage* beteiligt ist. Da dieses Projekt noch nicht existiert, wird ein neuer Datensatz in der Tabelle *t_proj* erzeugt.

⑬ Da das Projekt *Kundenumfrage* bereits existiert, wird kein neuer Datensatz für dieses Projekt angelegt.

Variablen in Stored Procedures

Variablen dienen zur Zwischenspeicherung von Werten, die Sie der Variablen zuweisen und später wieder auslesen können. In einer Prozedur sind verschiedene Variablentypen möglich.



Achten Sie bei der Definition von Variablen darauf, dass der Name, den Sie für die Variable wählen, nicht schon für eine Spalte, mit der Sie arbeiten möchten, vergeben ist. Eine Namensgleichheit kann zu Fehlfunktionen führen.

Eine häufig verwendete Form von Variablen sind die lokalen Variablen (im Beispiel *CreateProcedure.sql* Schritt ④). Sie werden zu Beginn der Prozedurdefinition nach dem Schlüsselwort `BEGIN` mit der Anweisung `DECLARE` deklariert. Sie sind nur innerhalb der Prozedur gültig.

Syntax

```
DECLARE name datentyp [DEFAULT wert];
```

- Die Deklaration der Variablen erfolgt in der Prozedurdefinition direkt nach dem Schlüsselwort `BEGIN`.
- Die Anweisung beginnt mit dem Schlüsselwort `DECLARE`. Danach folgt der gewünschte Name der Variablen.
- Anschließend muss ein Datentyp angegeben werden, der für die zu speichernden Daten passend ist. Hier sind alle Datentypen des Datenbanksystems erlaubt. Ein Standardwert kann über die `DEFAULT`-Anweisung angegeben werden.
- Lokale Variable gelten immer nur innerhalb des `BEGIN` … `END`-Blocks, in dem sie deklariert wurden.
- Variablen können auch in verschachtelten Blöcken verwendet werden.

© HERDT-Verlag Schulversion

Ergebnisse einer Abfrage mit SELECT INTO in Variablen speichern

Um die Ergebnisse einer Abfrage in Variablen speichern zu können, wird die SELECT-Abfrage um das Schlüsselwort INTO erweitert (im Beispiel *CreateProcedure.sql* Schritt ⑦). Danach können Sie für jedes Datenfeld eine Variable angeben, in der der entsprechende Wert gespeichert werden soll.

Syntax

```
SELECT ... INTO variable1, ...;
```

- Die Anweisung beginnt mit dem Schlüsselwort SELECT. Danach folgen die für die Abfrage benötigten Angaben.
- Am Ende der Abfrage werden das Schlüsselwort INTO und die Liste der Variablen angefügt. Für jedes Datenfeld der Ergebnismenge muss eine passende Variable angegeben werden. Die einzelnen Variablen werden durch Kommata getrennt.

IF-THEN-ELSE-Verzweigung

Im Ablauf einer Prozedur kann es oft notwendig sein, dass aufgrund einer Bedingung bestimmte Anweisungen ausgeführt oder auch nicht ausgeführt werden sollen (im Beispiel *CreateProcedure.sql* Schritt ⑧). Dieses Verhalten erreichen Sie mit der Alternative IF . . . THEN . . . ELSE. Der Anweisungsblock im THEN-Zweig wird immer ausgeführt, wenn die Bedingung erfüllt ist. Ist sie falsch, werden die Anweisungen im optionalen ELSE-Zweig ausgeführt.

Bei dieser Alternative wird stets ein Anweisungsblock ausgeführt. Es ist nicht möglich, dass beide Blöcke oder keiner von beiden ausgeführt wird.

Syntax der IF-THEN-ELSE-Anweisung

```
IF (bedingung) THEN
  anweisungsblock1
[ELSE
  anweisungsblock2]
END IF
```

- Die Anweisung beginnt mit dem Schlüsselwort IF. Danach folgt in runden Klammern die gewünschte Bedingung.
- Nach dem Schlüsselwort THEN folgt der erste Anweisungsblock. Er wird ausgeführt, wenn die Bedingung erfüllt ist.
- Optional kann mithilfe des Schlüsselworts ELSE ein zweiter Anweisungsblock angegeben werden, der ausgeführt wird, wenn die Bedingung nicht erfüllt ist.
- Die Anweisung muss mit den Schlüsselwörtern END IF abgeschlossen werden.

Die meisten Datenbanksysteme bieten viele weitere Möglichkeiten, Prozeduren zu steuern. MariaDB unterstützt z. B. zusätzlich zu den Befehlen im Beispiel folgende Anweisungen: CASE, LOOP, LEAVE, ITERATE, REPEAT und WHILE in Prozeduren. Nähere Informationen finden Sie in den jeweiligen Handbüchern.

© HERDT-Verlag

Prozedur ausführen

Möchten Sie eine Prozedur ausführen, verwenden Sie die Anweisung `CALL` (im Beispiel *CreateProcedure.sql* Schritt ⑫ und ⑬).

Syntax

```
CALL prozedurname (parameter1, ...);
```

- Die Anweisung beginnt mit dem Schlüsselwort `CALL`. Danach folgt der Name der auszuführenden Prozedur.
- Falls die Prozedur Übergabeparameter erwartet, müssen diese in der richtigen Reihenfolge und durch Kommata getrennt aufgeführt werden. Die runden Klammern sind dabei optional.

Selektierbare Prozedur starten

Datenbanksysteme, die selektierbare Prozeduren unterstützen, ermöglichen es, Prozeduren wie Tabellen in einer `SELECT`-Anweisung zu verwenden. Sie liefern eine Ergebnismenge zurück. Falls die Prozedur Übergabeparameter erwartet, müssen diese in runden Klammern angegeben werden. MariaDB unterstützt keine selektierbaren Prozeduren.

15.4 Übung

Prozeduren in MariaDB erstellen

Übungsdatei: -- **Ergebnisdatei: *Uebung.sql***

1. Erstellen Sie in der Datenbank *Uebungen* eine Prozedur *p_artikel_hinzufuegen*, die dem Lagerbestand
   in der Tabelle *t_lager* neue Artikel hinzufügt.
2. Die Prozedur soll als Parameter die Artikelnummer, die Stückzahl und den Preis erhalten. Verwenden Sie geeignete Namen und Datentypen.
3. Deklarieren Sie eine lokale Variable `aid` mit dem Datentyp `INTEGER`.
4. Prüfen Sie mit einer `SELECT`-Abfrage, ob der einzufügende Artikel bereits vorhanden ist, und speichern Sie das Abfrageergebnis auf der Variablen `aid`.
5. Wenn der Artikel schon im Lagerbestand ist, sollen Preis und Stückzahl des vorhandenen Artikels mit den neuen Werten aktualisiert werden.
6. Handelt es sich um einen neuen Artikel, soll er dem Lagerbestand hinzugefügt werden.
7. Testen Sie die Prozedur mit geeigneten Werten.

© HERDT-Verlag
Schulversion